Vagrant und VirtualBox unter Windows einrichten

Schritt für Schritt für Windows-User ohne Vorkenntnisse

Michael Schaar

30. September 2026

Inhaltsverzeichnis

Vagrant und VirtualBox unter Windows einrichten **2**
1. Voraussetzungen prüfen . . . 2
2. Git Bash installieren . . . 3
3. VirtualBox installieren . . . 3
4. Vagrant installieren . . . 4
5. Projekt herunterladen . . . 4
6. Virtuelle Maschine starten . . . 5
7. Einrichtung prüfen und Werkzeuge öffnen . . . 6
8. Ein Terminal in der VM öffnen . . . 7
9. ROS 2 kurz testen . . . 7
10. Kursmaterial und eigene Projekte einrichten . . . 8
11. VM anhalten oder entfernen . . . 9
Häufige Probleme unter Windows . . . 9
Weiteres Vorgehen . . . 11
Glossar . . . 11
Schlagworte . . . 12

Vagrant und VirtualBox unter Windows einrichten

Wirtschaftsinformatik und Kommandozeile – für viele eine Kombination, die im Studienplan nicht vorgesehen war. Diese Anleitung bringt euch trotzdem ohne Vorkenntnisse in Git, Vagrant oder ROS 2 (Robot Operating System 2 – Middleware-Framework für Roboter-Software, siehe Glossar) zum Ziel: das Robotik-Projekt aus dem Kursmaterial läuft am Ende auf eurem Windows-Rechner.

Drei Werkzeuge sind beteiligt, und jedes hat genau eine Aufgabe. Vagrant steuert die virtuelle Maschine (VM – ein Computer im Computer, isoliert vom Rest des Systems). VirtualBox führt sie aus. Git Bash liefert das Terminal, in dem ihr die Befehle eintippt. Mehr Bauteile braucht es nicht.

---

1. Voraussetzungen prüfen

Ihr braucht:

- einen Windows-PC mit 64-Bit-Prozessor
- aktivierte Hardware-Virtualisierung (Intel VT-x oder AMD-V)
- ausreichend freien Arbeitsspeicher und Festplattenspeicher

Die Projektbeschreibung nennt **8–10 GB freien Arbeitsspeicher** als Systemvoraussetzung für den Rechner, auf dem die VM läuft – nicht als Speicher, der der VM selbst zugewiesen wird. Windows braucht parallel dazu ebenfalls RAM, während die VM läuft. Wie viel Arbeitsspeicher der virtuellen Maschine konkret zugeteilt ist, steht im `Vagrantfile` (Konfigurationsdatei, die Vagrant vorgibt, wie die VM aussehen soll) des Projekts. Wer sich nicht sicher ist, ob der eigene Rechner das mitmacht, schaut dort nach, bevor er loslegt.

Zur Festplatte zwei getrennte Zahlen, die nichts direkt miteinander zu tun haben:

- Die virtuelle Festplatte ist standardmäßig auf **64 GB** ausgelegt. Das ist der Rahmen, den die VM maximal nutzen darf – kein tatsächlicher Verbrauch.
- Die Box-Datei (vorgefertigtes VM-Image, das beim ersten Start heruntergeladen wird) ist rund **18 GB** groß.

Wichtig dabei: Der Speicherplatz auf eurer Windows-Festplatte wird erst belegt, wenn die VM ihn tatsächlich beschreibt – die 64 GB sind also kein Betrag, der sofort komplett verbraucht wird, sondern eine Obergrenze, die mit der Zeit langsam gefüllt werden kann. Wer die Größe anpassen möchte, kann das laut Projektdokumentation direkt beim Start festlegen:

```
DISK_SIZE=40GB vagrant up
```

VirtualBox läuft grundsätzlich auch unter Windows Home. Trotzdem gibt es ein Konfliktpotenzial: Windows-Funktionen wie WSL 2, Docker Desktop oder „Speicherintegrität" nutzen teils denselben Virtualisierungsmechanismus wie Hyper-V und können VirtualBox ausbremsen oder am Start hindern. Microsoft dokumentiert dieses Verhalten selbst. Trotzdem gilt: Nichts vorsorglich abschalten. Nur wer tatsächlich auf den in Abschnitt „Häufige Probleme" beschriebenen Fehler stößt, muss nachjustieren – und auch dann erst nach kurzem Abwägen.

---

2. Git Bash installieren

1. Ladet **Git for Windows** von der offiziellen Git-Seite herunter.
2. Startet das Installationsprogramm.
3. Für diese Anleitung reichen die Standardoptionen.
4. Öffnet danach über das Startmenü **Git Bash**.

Git Bash ist eine Kommandozeilenumgebung für Windows, die Git mitbringt und alle Befehle dieser Anleitung ausführen kann.

---

3. VirtualBox installieren

1. Ladet VirtualBox für Windows von der offiziellen Oracle-Downloadseite herunter.
2. Startet das Installationsprogramm. Windows fragt eventuell nach Administratorrechten.
3. Bestätigt die Installation der benötigten Treiber.
4. Startet den Rechner neu, falls der Installer danach verlangt.

Installiert zunächst nur VirtualBox selbst. Das **VirtualBox Extension Pack** (Zusatzpaket mit erweiterten Funktionen, unter anderem USB-Unterstützung) braucht ihr für den normalen VM-Betrieb nicht. Laut Projektbeschreibung ist es nur nötig, wenn ihr USB-Geräte wie Arduino- oder ESP32-Boards an die VM durchreichen wollt.

Zur Versionswahl: Vagrant unterstützt nicht jede VirtualBox-Version gleichermaßen gut. Nach aktuellem Stand unterstützt Vagrant VirtualBox-Versionen bis einschließlich 7.2. Installiert nach Möglichkeit eine Version aus diesem unterstützten Bereich, statt reflexartig zur allerneuesten VirtualBox-Version zu greifen. Diese Angabe ändert sich mit neuen Vagrant-Releases – prüft die offizielle Vagrant-Dokumentation zum Zeitpunkt eurer Installation erneut.

---

4. Vagrant installieren

1. Ladet Vagrant für Windows von der offiziellen HashiCorp-Seite herunter.
2. Startet den Installer und folgt den Schritten.
3. Startet Windows neu, wenn ihr dazu aufgefordert werdet.

Danach die Kontrolle. Öffnet **Git Bash** und gebt ein:

```
vagrant --version
VBoxManage --version
```

Beide Befehle sollten eine Versionsnummer ausspucken. Meldet Windows, ein Befehl sei nicht gefunden: Git Bash neu starten. Bleibt der Fehler, stimmt etwas mit der Installation nicht – nochmal installieren.

---

5. Projekt herunterladen

In Git Bash:

```
git clone https://codeberg.org/kraeml/robotics-lab-vm.git
cd robotics-lab-vm
```

`git clone` (Kommando zum Herunterladen eines kompletten Projekt-Repositorys) lädt die Projektdateien herunter, `cd` (change directory) wechselt in den neuen Ordner.

Wer keine Kommandozeile zum Herunterladen mag, lädt stattdessen die ZIP-Datei von der Projektseite und entpackt sie vollständig. Danach in diesem Ordner: Rechtsklick auf eine freie Stelle im Explorer, Git Bash öffnen.

---

6. Virtuelle Maschine starten

Git Bash muss im Projektordner geöffnet sein. Dann:

```
vagrant up
```

Beim ersten Start lädt Vagrant die VM-Box herunter – mehrere Gigabyte, je nach Verbindung ein Kaffee oder zwei. Anschließend erstellt Vagrant die VM und startet sie in VirtualBox.

Bei jedem weiteren Start reicht derselbe Befehl. Vagrant merkt sich den Zustand anhand der Projektdateien.

Falls die Standardadresse belegt ist: Die VM nutzt standardmäßig `192.168.33.10`. Wer bereits eine andere Vagrant-VM mit derselben Adresse betreibt, bekommt einen Konflikt. Laut Projektbeschreibung lässt sich das beim Start umgehen:

```
VM_IP=192.168.33.20 vagrant up
```

Verwendet dann in allen folgenden Schritten diese Adresse statt `192.168.33.10`.

Nach Änderungen am `Vagrantfile`: Manche Einstellungen wie Netzwerkoptionen lassen sich mit

```
vagrant reload
```

auf eine bereits laufende VM übernehmen, ohne sie komplett neu aufzusetzen. Das gilt aber nicht für jede Änderung. Bei tiefgreifenden Anpassungen – etwa der Festplattengröße – ist die Projektdokumentation maßgeblich: manche Werte lassen sich nachträglich gar nicht mehr ändern und erfordern stattdessen `vagrant destroy` mit anschließendem `vagrant up`.

---

7. Einrichtung prüfen und Werkzeuge öffnen

Nach dem Start erscheint ein Fenster mit Ubuntu-Desktop. Anmeldedaten, falls nötig:

- **Benutzername:** `vagrant`
- **Passwort:** `vagrant`

Öffnet anschließend im Browser **auf Windows** – nicht im VM-Fenster – diese Adressen:

Werkzeug	Adresse	Anmeldung
code-server (VS Code im Browser, siehe Glossar)	`http://192.168.33.10:8080`	Passwort: `change_me`
JupyterLab (browserbasierte Entwicklungsumgebung für Python-Notebooks)	`http://192.168.33.10:8888`	laut Projektbeschreibung ohne Passwort

Die Zugangsdaten erscheinen beim Start zusätzlich im Git-Bash-Terminal. Lädt eine Adresse nicht: warten, bis der VM-Start abgeschlossen ist, und prüfen, ob eine Firewall dazwischenfunkt.

> **Sicherheit:** Ändert das Standardpasswort von code-server, bevor ihr die Umgebung in einem Netzwerk verwendet, dem ihr nicht vertraut. Dafür in der VM (per `vagrant ssh` oder Terminal in code-server) folgende Datei bearbeiten:
>
> ```
> nano ~/.config/code-server/config.yaml
> ```
>
> Passwort dort ändern, speichern, und den Dienst neu starten:
>
> ```
> sudo systemctl restart code-server@vagrant
> ```
>
> JupyterLab läuft ohne Passwort – betreibt die VM deshalb nicht ungeschützt in einem fremden oder öffentlich erreichbaren Netzwerk. Klingt nach Kleinkram, ist aber der Unterschied zwischen „nur ich sehe mein Notebook” und „das ganze Café auch”.

---

8. Ein Terminal in der VM öffnen

Zwei Wege führen zum Ziel.

Im Browser: code-server öffnen, Menü **Terminal → New Terminal**. Die ROS-2-Umgebung ist dort bereits eingerichtet.

Mit Git Bash: Git Bash im Projektordner öffnen:

```
vagrant ssh
```

Ihr seid jetzt per Kommandozeile in der VM. Kurzer Check, ob ROS 2 korrekt eingerichtet ist:

```
echo $ROS_DISTRO
```

Erwartet wird `jazzy` (Codename der im Projekt verwendeten ROS-2-Version). Mit `exit` oder **Strg + D** geht's zurück nach Windows.

---

9. ROS 2 kurz testen

In einem VM-Terminal aktive Themen anzeigen:

```
ros2 topic list
```

Für einen echten Test öffnet ihr zwei Terminals in der VM. Im ersten ein Sender:

```
ros2 run demo_nodes_cpp talker
```

Im zweiten der Empfänger:

```
ros2 topic echo /chatter
```

Tauchen im zweiten Terminal laufend Nachrichten auf, funktioniert die Kommunikation zwischen den Programmen – Grundprinzip der gesamten ROS-Architektur.

---

10. Kursmaterial und eigene Projekte einrichten

Zwei Ordner in der VM sind für die eigentliche Arbeit relevant.

Notebooks des Kursmaterials: Das Unterrichtsmaterial liegt zunächst schreibgeschützt unter `/vagrant/notebooks`. Legt euch einmalig eine eigene Arbeitskopie an, damit ihr speichern könnt, ohne das Git-Repository zu verändern:

```
mkdir -p ~/notebooks && cp -a /vagrant/notebooks/. ~/notebooks/
```

Arbeitet danach unter `~/notebooks`. Wichtig beim erneuten Ausführen dieses Befehls, etwa nach einem `git pull` auf Windows: `cp -a` überschreibt vorhandene Dateien mit gleichem Namen, **löscht aber keine Dateien**, die im Original inzwischen entfernt wurden. Eure Kopie kann dadurch veraltete Notebooks enthalten, die im Kursmaterial gar nicht mehr existieren. Sichert eigene Notizen vorher separat, etwa durch Kopieren in einen eigenen Unterordner – der erneute Kopierbefehl überschreibt sonst kommentarlos alles, was ihr in einer gleichnamigen Datei verändert habt.

Eigene ROS-Projekte: Neue Pakete gehören nach `~/workspace/src`:

```
mkdir -p ~/workspace/src && cd ~/workspace/src
ros2 pkg create mein_paket --build-type ament_python
cd ~/workspace && colcon build && source install/setup.bash
```

`colcon build` (Build-Werkzeug für ROS-2-Workspaces, kompiliert und verknüpft alle Pakete im Ordner) übersetzt euren Code, `source install/setup.bash` macht ihn in der aktuellen Terminal-Sitzung nutzbar.

Der Ordner `~/workspace` ist mit eurem Windows-Rechner geteilt. Eigene Projekte landen also automatisch auch im Projektordner unter Windows und überleben selbst das Löschen der VM (siehe Abschnitt 11).

Vorgefertigte Beispielprojekte aus dem Kurs kopiert ihr vor dem Bauen ebenfalls dorthin:

```
cp -a /vagrant/examples/mein_beispiel ~/workspace/src/
```

---

11. VM anhalten oder entfernen

Diese Befehle laufen in **Git Bash auf Windows**, aus dem Projektordner heraus:

Befehl	Wirkung
`vagrant suspend`	Friert die VM ein; später fortsetzbar.
`vagrant halt`	Fährt die VM herunter; Daten bleiben erhalten.
`vagrant up`	Startet die VM erneut.
`vagrant reload`	Übernimmt bestimmte Konfigurationsänderungen, ohne die VM neu zu erstellen.
`vagrant destroy`	Entfernt die VM vollständig.

Zum Datenverbleib bei `vagrant destroy`: Der geteilte Ordner `~/workspace` liegt technisch im Projektordner auf Windows (`workspace/` neben dem `Vagrantfile`). Eigene Projekte darin überstehen also auch ein `vagrant destroy` – solange ihr den Windows-Projektordner nicht selbst löscht. Alles, was ihr dagegen nur innerhalb der VM gespeichert habt – etwa direkt unter `~/notebooks` oder in einem separaten, nicht geteilten Arbeitsbereich wie `~/ros2_ws` – **wird mit der VM gelöscht**. Sichert solche Dateien vorher, wenn ihr sie behalten wollt.

Häufige Probleme unter Windows

`vagrant` oder `VBoxManage` wird nicht gefunden Git Bash schließen und neu öffnen. Hilft das nicht, das fehlende Programm neu installieren und Windows neu starten.

Die VM startet nicht oder läuft auffällig langsam Erst prüfen, ob Virtualisierung in den BIOS-/UEFI-Einstellungen aktiviert ist. VirtualBox kann außerdem durch Hyper-V oder davon abhängige Windows-Funktionen ausgebremst werden – muss aber nicht. Neuere VirtualBox-Versionen kommen mit aktivem Hyper-V teilweise klar, andere nicht. Nicht vorsorglich etwas abschalten – erst reagieren, wenn tatsächlich der folgende Fehler auftritt.

Fehler wie `VT-x is not available` oder `VERR_VMX_NO_VMX` Dieser Fehler *kann* bedeuten, dass Hyper-V die Virtualisierung für VirtualBox blockiert – er beweist es aber nicht automatisch.

Prüft zuerst, ob Virtualisierung im BIOS/UEFI überhaupt aktiviert ist (siehe oben); ohne das hilft auch das Abschalten von Hyper-V nichts.

Ist das erledigt und der Fehler bleibt bestehen, kommt oft WSL2, Docker Desktop oder die Windows-Sandbox als heimlicher Hyper-V-Aktivator infrage. Bevor ihr eingreift: Überlegt, ob ihr eine dieser Funktionen parallel braucht. Das Abschalten von Hyper-V deaktiviert sie ebenfalls, nicht nur den Konflikt mit VirtualBox. Microsofts eigene Dokumentation zu diesem Konflikt beschreibt die Abwägung im Detail – ein Blick dorthin lohnt sich, bevor ihr den Befehl ausführt.

Wer die Konsequenzen abgewogen hat, öffnet die Eingabeaufforderung als Administrator:

```
bcdedit /set hypervisorlaunchtype off
```

Windows neu starten. Rückgängig machen:

```
bcdedit /set hypervisorlaunchtype auto
```

Die Browser-Adressen laden nicht Warten, bis Vagrant den Start beendet hat (die Zugangsdaten-Übersicht muss im Terminal erschienen sein). Danach prüfen, ob eine Firewall blockiert, und – falls `VM_IP` angepasst wurde – die richtige Adresse verwenden.

Vagrant meldet: „A VirtualBox machine with the name 'robotics' already exists" In VirtualBox existiert bereits eine Maschine mit diesem Namen, meist ein Überbleibsel eines früheren Versuchs.

1. Vorhandene Maschinen anzeigen:

```
VBoxManage list vms
```

2. Alte Maschine nicht mehr gebraucht? Löschen (entfernt auch die virtuelle Festplatte):

```
VBoxManage unregistervm "robotics" --delete
```

3. `vagrant up` erneut ausführen.

Liegen in der alten Maschine noch Daten, die ihr behalten wollt: umbenennen statt löschen.

```
VBoxManage modifyvm "robotics" --name robotics-alt
```

Danach ebenfalls `vagrant up`. Auf `robotics-alt` lässt sich später zugreifen oder es irgendwann entfernen.

Verwaiste Vagrant-Einträge aufräumen:

```
vagrant global-status --prune
```

Weiteres Vorgehen

Läuft die VM, orientiert ihr euch für den Kursablauf an der Projektdokumentation zum Unterrichtskonzept – dort steht, wie die Umgebung phasenweise für projektorientiertes Arbeiten genutzt wird. Probiert die einzelnen Schritte am besten gleich selbst aus, statt die Anleitung nur zu lesen und zu hoffen, dass es beim ersten eigenen Versuch schon klappt.

Glossar

- **Box:** Vorgefertigtes VM-Image, das Vagrant beim ersten Start herunterlädt und als Basis für die virtuelle Maschine verwendet.
- **code-server:** Webbasierte Version des Editors VS Code, läuft im Browser statt als lokale Anwendung.
- **colcon build:** Build-Werkzeug für ROS-2-Workspaces, kompiliert und verknüpft alle Pakete in einem Projektordner.
- **git clone:** Kommando zum Herunterladen eines kompletten Projekt-Repositorys von einem Server auf den eigenen Rechner.
- **Hyper-V:** Windows-eigene Virtualisierungstechnik, die mit VirtualBox um dieselben Systemressourcen konkurrieren kann.
- **JupyterLab:** Browserbasierte Entwicklungsumgebung, vor allem für interaktive Python-Notebooks verwendet.
- **ROS 2 (Robot Operating System 2):** Middleware-Framework für Roboter-Software; stellt Kommunikation zwischen Programmteilen (“Nodes”) bereit.
- **Vagrantfile:** Konfigurationsdatei, in der festgelegt ist, wie die von Vagrant verwaltete VM aussieht (z. B. zugewiesener Arbeitsspeicher, Netzwerkadresse).

- **VirtualBox Extension Pack:** Zusatzpaket für VirtualBox mit erweiterten Funktionen, unter anderem USB-Unterstützung für angeschlossene Geräte.
- **VM (virtuelle Maschine):** Ein vollständiger, isolierter Computer, der als Software innerhalb eines anderen Betriebssystems läuft.

Schlagworte

Vagrant, VirtualBox, Windows, ROS2, Git-Bash, Virtualisierung, Robotik, code-server, JupyterLab, Hyper-V, Tutorial, Wirtschaftsinformatik, Ubuntu, Kursmaterial, Entwicklungsumgebung